

CAPSTONE DEFENCE PREPARATION

# SuppliWise Web3 Layer — Developer & Defense Manual

SuppliWise · Web3 / Blockchain Layer

# Contents

---

## Table of contents

2.1 Canonical JSON — `stableStringify`

2.2 Payload hash — `hashPayload`

2.3 Transaction hash — `normalizeTx`

2.4 Decentralized identity — ed25519

2.5 Encryption + content addressing — AES-256-GCM

The one thing you must get right

3.1 Mining a block — real output

3.2 What the block hash actually covers

Tamper evidence — demonstrated

The limitation — know this cold

3.3 Three verification strategies

3.4 Genesis and the chain's starting point

The architectural consequence — understand this

4.1 Money movement — the parts that are genuinely correct

The one gap

4.2 Idempotency — the pattern to point at

Feature 1 — Immutable supply chain tracking

Feature 2 — Verifiable certifications

Features 3 & 4 — Escrow commerce and P2P marketplace

The escrow lifecycle

Feature 5 — Decentralized identity

Feature 6 — User-owned health ledger

Feature 7 — Data sovereignty rewards

Feature 8 — Decentralized encrypted storage

Feature 9 — WELL token rewards

Every reward, with real arithmetic

Check-in streak — real output

Feature 10 — Achievement NFTs

Feature 11 — Staking for premium access

Feature 12 — Blockchain loyalty program

Feature 13 — DAO governance

The tally — real output

Feature 14 — Community knowledge base

Feature 15 — Decentralised dispute resolution

Real verdicts

Feature 16 — Interoperable health profile

Feature 17 — Verifiable AI recommendations

Features 18, 19, 20 — Ecosystem

The oracle feeds — be blunt about these

Scenario A — A user's first 15 minutes

Scenario B — Two users, one marketplace order

Scenario C — A dispute

Scenario D — Governance, including the failure mode

7.1 DAO parameters (`models/Web3.js:109-128`)

7.2 System wallets

7.3 Status enums

7.4 Chain operation reference

Tier 1 — Fundamentals

Tier 2 — Tamper evidence

Tier 3 — Tokens and governance

Tier 4 — Security and privacy

Tier 5 — Engineering

# SuppliWise Web3 Layer — Developer & Defense Manual

---

**Who this is for:** you, standing in front of a panel. This document explains what the blockchain layer actually does, with real output from the real code, so you can explain it without bluffing.

**Every number and hash in this manual was produced by running the project's own**

`server/blockchain/crypto.js` and `server/blockchain/rules.js` on Node v24.19.0. Nothing here is invented. The verification script is reproduced in Appendix B so a panel member can re-run it.

**Read Part 1 and Part 8 before anything else.** Part 1 is the 60-second explanation. Part 8 is the question bank — that is what you will actually be graded on.

---

## Table of contents

| Part | Contents                                                       |
|------|----------------------------------------------------------------|
| 1    | The 60-second mental model                                     |
| 2    | Cryptography you actually use (4 primitives, with real output) |
| 3    | The ledger: how a block is built and verified                  |
| 4    | The state engine: database-first, then anchor                  |
| 5    | The 20 features, one by one                                    |
| 6    | Worked end-to-end scenarios                                    |
| 7    | Reference tables                                               |
| 8    | The question bank — 25 questions with honest answers           |
| 9    | Vocabulary cheat sheet                                         |
| 10   | Live demo script                                               |
| A    | Known limitations, stated plainly                              |
| B    | Verification script                                            |

---

## Part 1 — The 60-second mental model

---

If you only memorise one thing, memorise this.

**SuppliWise runs a private, append-only, SHA-256 hash-chained audit ledger, stored in our own MongoDB, that records every Web3 state change as a mined block. Real cryptography — real SHA-256, real ed25519 signatures, real AES-256-GCM — but the ledger lives inside our server. It is a tamper-evident audit log, not a distributed consensus network. There is one writer, one database, and no mining competition. We built it that way deliberately: it gives users and auditors a verifiable cryptographic trail without the cost or the operational burden of running a public chain.**

Then, if pressed, add the two sentences that show you understand the boundary:

"The tamper-evidence comes from hash-chaining — each block commits to the previous block's hash, so editing an old block forces you to re-mine every block after it. The limitation is that we are the custodian: whoever controls this database could re-mine the whole chain from scratch. That is a scope boundary, not an implementation bug, and I've documented it in Appendix A."

**Why this framing wins.** The alternative — calling it "SuppliWise Mainnet" — is a claim a panel member can falsify in one question. This framing is defensible, accurate, and demonstrates you know exactly where the system ends.

## Part 2 — Cryptography you actually use

Four primitives. Nothing hand-rolled, nothing fake. This is the strongest part of the module and you should say so.

All output below is real, from the project's own code.

### 2.1 Canonical JSON — `stableStringify`

**The problem it solves.** `JSON.stringify` preserves key insertion order. Two objects that are *identical in content* can produce different bytes depending on how they were built:

```
Input A: {"amount":25,"kind":"checkin","to":"0xabc"}
Input B: {"to":"0xabc","kind":"checkin","amount":25}

stableStringify(A) = {"amount":25,"kind":"checkin","to":"0xabc"}
stableStringify(B) = {"amount":25,"kind":"checkin","to":"0xabc"}
A === B ?           true

Naive JSON.stringify(A) === JSON.stringify(B) ? false    <-- the bug this prevents
```

**Why it matters for a hash.** A hash is computed over bytes. If the bytes can differ for the same logical content, then the same transaction can have two different valid hashes — and the tamper-evidence property collapses. Sorting keys recursively (and dropping `undefined`) makes hashing deterministic.

Source: `server/blockchain/crypto.js:16-23`

### 2.2 Payload hash — `hashPayload`

```
hashPayload(p) = sha256(stableStringify(p))
```

```
payload          = {"amount":100,"reason":"Wallet welcome bonus"}
stableStringify  = {"amount":100,"reason":"Wallet welcome bonus"}
dataHash         = e768dba4a641a06c273c2bf9d64efe876e4debea89f34c489329226adfab4e2a
verify by hand   = e768dba4a641a06c273c2bf9d64efe876e4debea89f34c489329226adfab4e2a
```

The second line is the important one: anyone can reproduce that digest with a one-line script. **Say this** — "you can verify our payload hashes with any SHA-256 tool; we don't ask you to trust us."

Source: `crypto.js:33-35`

## 2.3 Transaction hash — `normalizeTx`

```
type      = reward:welcome_bonus
actor     = 0xDEMO
data      = {"amount":100,"reason":"Wallet welcome bonus"}
dataHash  = e768dba4a641a06c273c2bf9d64efe876e4debea89f34c489329226adfab4e2a
txHash    = c3659540ead34a462d46f3f45a7b146288db926d78f573cbcbfbee698aa9f0f3
```

The formula is:

```
txHash = sha256(stableStringify({ type, actor, dataHash, timestamp }))
```

Note the structure: the readable `data` never enters `txHash` directly. It enters **only** as `dataHash`. This is what lets us put personal health payloads on a public chain — the payload is stored as a digest, not in the clear.

**This is the single most defensible design decision in the module.** `headerHash` (§3.2) later reads `txHash` strings without re-hashing payloads, which is what makes verification fast. The digest indirection is deliberate and load-bearing.

Source: `ledger.js:70-76`

## 2.4 Decentralized identity — ed25519

```
did      = did:suppliwise:<userId>
publicKey = YrYXw/YFZWkIFu7F9huAvFAwFsX1bqLHZLPmDrdtq/o=
address   = 0x6ce97511bfaa7ff1a46b259b164cf665078443c9

address is deterministic from the public key ? true

sign(message)      = 4HLIx9C5G+hZGafBB32EAdJZSws7GYs/K0P+OB20k1Ek...
verify(pub, msg, sig) = true
verify(pub, TAMPERED) = false
```

How the address is derived (`:57-59`):

```
address = "0x" + sha256("sw:addr:" + base64PublicKey).slice(0, 40)
```

That is a real ed25519 keypair (`crypto.generateKeyPairSync('ed25519')`), and the last two lines demonstrate the property that matters: **a valid signature verifies; a tampered message fails.**

Used by feature 16 to issue a signed, portable health credential — the server signs with the user's key so a third party can verify it without an account.

⚠ **Be accurate about this:** the private key is generated server-side and stored encrypted (AES-256-GCM under `JWT_SECRET`), and the server signs on the user's behalf. So the *signature is real and verifiable*, but this is **not** self-custody — the user cannot revoke the server's ability to sign. If a panel member says "that's self-custody," correct it politely and move on. Full detail in Appendix A, item 5.

Source: `crypto.js:56-99`, `engine.js:136-144`

## 2.5 Encryption + content addressing — AES-256-GCM

```
plaintext = {"ageBand": "20-29", "adherenceBand": "60-29"}
iv = YuwOdL8GtVkiYurC (12 random bytes per call)
tag = j5RJ8JPjEF60HkVmdXDRLA==
decrypt = {"ageBand": "20-29", "adherenceBand": "60-29"}
decrypt with wrong key throws (auth tag mismatch) ? YES
```

GCM gives three things at once: confidentiality, integrity, and authentication of the ciphertext. The wrong-key failure is the auth tag doing its job.

### The one thing you must get right

The code comment at `crypto.js:123-124` says:

*"the same content always maps to the same CID (tamper-evident by design)"*

**That claim is false, and here is the proof from our own run:**

```
cid of first pin = bafyuurbli6o65g7sdxbwzg4apprqi6r4s6b7oqcxfatppuygsrgya
cid of second pin = bafyhu5n3shzwpfzwx3glpcpdev7k7z33oxv3f7f5eruqvafvnbaupq
identical plaintext, same CID ? false
```

Because `encrypt()` generates a **fresh random IV on every call**, identical plaintext produces different ciphertext, and `contentId()` hashes the ciphertext.

**If a panel member asks "what does the CID identify?"**, the correct answer is: *"It identifies a specific encrypted blob — a specific pin operation. It is tamper-evident for that ciphertext; it is not a deduplication key for the plaintext."* Never claim content-addressing of plaintext.

Source: `crypto.js:102-128`

## Part 3 — The ledger: how a block is built and verified

### 3.1 Mining a block — real output

```
difficulty = 3
winning nonce = 5210    (5211 attempts, 79.7 ms)
block hash    = 000a84b3fa93fcc7318dfc2c30ec6c175fb5f14d0bab2d31d460419dcec72038
leading zeros  = 000    <-- satisfies difficulty 3
target odds   = 1 in 4,096
```

The rule is simple ( `ledger.js:26-28` ): the block hash must begin with `DIFFICULTY` zeros.

**The mining loop** ( `ledger.js:61-67` ):

```
for (let nonce = 0; nonce <= MAX_NONCE; nonce += 1) {
  const hash = headerHash({ ...header, nonce });
  if (meetsDifficulty(hash)) return { nonce, hash };
}
```

5211 attempts in 80ms. At difficulty 3 that's expected — 1 in 4,096. There is **no difficulty adjustment** and **no block reward**: this is not a competitive mining system. It's a deliberate, bounded computational cost that makes appending a block non-trivial.

**If asked about performance honestly**: 80ms per block on the server's event loop is real, and at higher difficulty settings it would become a DoS lever. The code bounds it with `MAX_NONCE` for exactly that reason ( `ledger.js:32-36` ). Say that — it shows you understand the tradeoff.

### 3.2 What the block hash actually covers

```
// ledger.js:21-24 - verbatim
function headerHash({ index, timestamp, prevHash, nonce, txs }) {
  const txPart = (txs || []).map((t) => t.txHash).join('|');
  return sha256Hex(stableStringify({ index, timestamp, prevHash, nonce, txPart }));
}
```

So the block hash commits to:

| Field                  | Meaning                                              |
|------------------------|------------------------------------------------------|
| <code>index</code>     | position in the chain                                |
| <code>timestamp</code> | when                                                 |
| <code>prevHash</code>  | the previous block's hash — <b>this is the chain</b> |
| <code>nonce</code>     | the mining work done                                 |
| <code>txPart</code>    | the <b>hash strings</b> of the transactions          |

Tamper evidence — demonstrated

Change one field, same nonce:

```
timestamp 1700000000000 -> nonce 5210 => 000a84b3fa93ffcc7318dfc2c30ec6c175fb5f14d0bab2d31d460419dcec72038
timestamp 1700000000001 -> nonce 5210 => 328f6ea2f0f9ce39fd25f1bbc17698649c69949193f1eb67db36acd208761fe0
hashes equal ? false
still meets difficulty ? false
```

**What this proves:** the stored `hash` no longer matches a recomputation, so `checkBlock` returns `'hash mismatch'` and the chain reports invalid at that height.

The limitation — know this cold

`txPart` contains only `t.txHash` strings. The readable `txs[].data` is excluded, and `VERIFY_PROJECTION` (`ledger.js:46`) explicitly projects it out.

Our own run:

```
block hash covers      : index, timestamp, prevHash, nonce, and txs[].txHash STRINGS ONLY
block hash does NOT cover: txs[].data (the readable payload)
change txs[0].data to TAMPERED, header hash still => unchanged
```

**Honest framing if asked:** *""The block hash commits to transaction identities via their hashes, and each transaction hash commits to its payload via `dataHash`. So the chain binds payloads indirectly. However, because `txs[].data` isn't itself inside the header hash, someone with direct database write access could edit a payload's readable text without invalidating the chain. Closing that would mean hashing the payload into the header — it's a three-line change and it's the first item on my list in Appendix A."\**

Do not claim payload-level immutability. That is the fastest way to get caught.

3.3 Three verification strategies

| Method                                   | Cost            | When it runs                              | What it catches                               |
|------------------------------------------|-----------------|-------------------------------------------|-----------------------------------------------|
| <code>audit()</code> — full re-hash      | O(entire chain) | Every 5 min, <code>?full=1</code>         | Edits to <b>any</b> block, including old ones |
| <code>verifyTail()</code> — incremental  | O(new blocks)   | Default on <code>GET /chain/verify</code> | Breaks at or after the checkpoint             |
| <code>checkBlock()</code> — single block | O(1)            | Inside both of the above                  | One block's linkage + hash + difficulty       |

`checkBlock` (`ledger.js:81-87`) is the shared validator — deliberately extracted so the two walks can't drift in what they consider valid:

```
function checkBlock(block, expectedPrev) {
  if (block.prevHash !== expectedPrev) return 'prevHash mismatch';
  const recomputed = headerHash(block);
  if (recomputed !== block.hash) return 'hash mismatch';
  if (!meetsDifficulty(block.hash)) return 'difficulty target not met';
  return null;
}
```

The checkpoint system is genuinely good engineering and worth demonstrating:

- `checkpoint` — how far integrity is *proven*. The incremental walk extends it.
- `verifiedAt` / `verifiedChecked` — when the **deep** audit last ran, and how many blocks it

The comment at `ledger.js:167-173` explains why these are tracked separately: only a deep audit advances them, because the incremental walk never re-examines the prefix. Letting it overwrite would make the monitor claim a full re-hash it never performed. That's a subtle correctness distinction a panel will respect.

The **pending: true** verdict. If the deep audit exceeds `VERIFY_BUDGET_MS` (5s), `verify()` returns `{ valid: true, pending: true }` rather than a false alarm. The design rationale (`ledger.js:322-324`): *"a check that has not come back yet is unknown, not broken."* That's correct reasoning — but be ready to explain that `pending` means *unproven*, not *proven valid*.

### 3.4 Genesis and the chain's starting point

Created on first boot only (`ledger.js:121-131`):

```
type: 'genesis', actor: 'sw:system',
data: { network: 'suppliwise-mainnet', difficulty: 3, launchedAt: Date.now() }
prevHash = '0'.repeat(64) // 64 zeros
```

The 64-zero `prevHash` is the standard convention: block 0 points at nothing.

## Part 4 — The state engine: database-first, then anchor

Read `engine.js:18-26` — it states the contract explicitly:

*"mutate the database FIRST (it is the source of truth for balances/records), then ANCHOR the change on-chain for the tamper-evident audit trail. An anchor failure is logged, never able to roll back or fail the business operation — availability over perfect atomicity, with the DB remaining authoritative."*

And `anchor()` (`:99-107`) never throws:

```
try { return await ledger.append([...]); }
catch (err) { console.error('[web3 anchor]', type, err.message);
  return { index: -1, hash: '', txs: [] }; }
```

## The architectural consequence — understand this

| Question                                        | Answer                                                                                                  |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| If the DB write succeeds but the anchor fails?  | Feature succeeds; <code>txHash</code> is <code>''</code> and <code>blockIndex</code> is <code>-1</code> |
| If the anchor succeeds but the DB write failed? | Impossible — DB goes first                                                                              |
| Is the blockchain the source of truth?          | <b>No.</b> MongoDB is                                                                                   |
| Is the chain an audit trail?                    | <b>Yes — a best-effort one</b>                                                                          |

If a panel member asks "is the database or the chain authoritative?", say "the database, by design and documented in `engine.js:18-26`. We chose availability over cross-store atomicity. The consequence is that an anchor can be dropped while the business operation succeeds — a token could carry `blockIndex: -1`. That's the honest tradeoff and it's in Appendix A."

## 4.1 Money movement — the parts that are genuinely correct

This is where you should sound **confident**, because the code is right.

**Debit is conditional** (`engine.js:232`) — one atomic operation, no read-then-write:

```
Wallet.findOneAndUpdate({ address, balance: { $gte: value } }, { $inc: { balance: -value } })
```

If funds are insufficient the filter simply doesn't match. **Double-spend is structurally impossible here.** There's no window between reading the balance and writing it back.

**The same pattern protects staking and accrual** (`:277`, `:292`, `:311-315`) — `accrueStake` even guards on the exact `lastStakeAccrualAt` timestamp it read, so two concurrent claims cannot both pay out.

**Reward idempotency is a unique index**, not application code (`models/Web3.js:337`):

```
rewardEventSchema.index({ user: 1, kind: 1, refId: 1 }, { unique: true });
```

`grantReward` catches `E11000` and returns `{ alreadyClaimed: true }`. A double-click or a retried request **cannot** mint twice — the database refuses the second insert.

### The one gap

`transfer()` (`:242-268`) is a hand-rolled two-phase commit:

```
const debited = await Wallet.findOneAndUpdate({...balance: {$gte: value}}, {$inc: {balance: -value}});
if (!debited) throw INSUFFICIENT;
const credited = await Wallet.findOneAndUpdate({address: to}, {$inc: {balance: value}}).catch(() => null);
if (!credited) { await Wallet.updateOne({address: from}, {$inc: {balance: value}}); throw NO_WALLET; }
```

The comment calls it "Atomic two-sided transfer." **It is not atomic** — there are two writes. If the process dies between them, the rollback never runs and the tokens are gone from the sender with no compensating credit.

**Honest answer if asked:** "Debit, credit, stake and reward accrual all use single conditional atomic updates and are correct. `transfer()` is two writes with a compensating rollback, so it's atomic in the normal case but not across a process crash. The fix is a MongoDB transaction or closing the client handle after the credit. It's item 4 in Appendix A."

## 4.2 Idempotency — the pattern to point at

Five unique compound indexes carry the weight:

| Index                               | Guarantees                                               |
|-------------------------------------|----------------------------------------------------------|
| <code>{ user, kind, refId }</code>  | one reward payout per (user, type, reference)            |
| <code>{ owner, kind }</code>        | one NFT of each kind per user — soulbound, no duplicates |
| <code>{ user, assessmentId }</code> | one AI-proof anchor per assessment                       |
| <code>{ user, trial }</code>        | one consent record per trial                             |
| <code>{ order }</code>              | one dispute per order                                    |

These are database-level invariants. They hold regardless of application bugs, race conditions, or retries. **When a panel member asks how you prevented double-spending, answer with indexes and conditional updates — not with "we check first."**

## Part 5 — The 20 features

Every feature is DELUXE tier ( `minTier: 'monthly'` ) unless noted. The gate is `server/routes/web3/guards.js` → `requireFeature()`, enforced at the router, not hidden in the UI.

### Feature 1 — Immutable supply chain tracking

**What it is.** A manufacturer registers a batch, then appends journey steps. Each step is anchored to the chain. A consumer scans a QR code and reads the journey back.

**How it works.** Seven fixed steps, strictly forward-only ( `models/Web3.js:152` ): `raw-sourcing` → `manufacturing` → `lab-testing` → `quality-release` → `distribution` → `retail` → `delivered`

Each step creates one block via `engine.anchor('supply:step', ...)` carrying only `{ code, step, at, prev }`.

**Public verification** — `GET /api/web3/verify/:code` requires no login. It looks up the batch's step hashes, finds their blocks, recomputes each header hash, and reports a proof object.

**Worked example.** Batch `SW-A1B2C3D4`, brand "SunWell Nutrition":

- `POST /supply/batches` → creates batch, auto-adds step 1 ( `raw-sourcing` ), anchors → **block N**
- `POST /supply/batches/:id/events` × 6 more → **blocks N+1 ... N+6**
- `GET /verify/SW-A1B2C3D4` → returns 7 steps + `valid: true` + `height: N+6`

**What to say:** "Each step is a separate mined block, so a step added later cannot change an earlier one without invalidating the chain from that point forward."

**What NOT to say:** don't claim the readable step text (location, notes) is on-chain — only `{code, step, at, prev}` is. And acknowledge that batch creation is self-attested by one account (Appendix A, item 3).

## Feature 2 — Verifiable certifications

Lab reports and certifications are anchored the same way (`engine.anchor('cert:anchor', ...)`).

Certification types (`models/Web3.js:168`): `lab-report`, `organic`, `non-gmo`, `third-party`, `gmp`, `other`.

**The result hash** (`supply.js:282-284`) is accepted if it matches `/^[a-f0-9]{64}|bafy[a-z2-7]+$/i`, otherwise derived from `hashPayload({ code, type, name, issuer, fileId })`.

**Honest framing:** *"We anchor a hash of the certificate metadata. There is no file upload pipeline — the client supplies a CID, and we don't verify the referenced document exists. This proves a *certificate claim was made and has not changed*, not that a lab report is genuine. Real attestation needs a regulator counter-signature, which is future work."*

⚠ **Critical for your defense:** the seeded demo data names **Eurofins Labs, USP, Informed Sport, and USDA Organic** (`blockchain/seed.js:326-367`). These are **fabricated demo labels**. If a panel member spots them, say so first — *"those are synthetic demo records, and I've already flagged replacing them with obviously-fake labels."* Never let them discover it.

## Features 3 & 4 — Escrow commerce and P2P marketplace

### The escrow lifecycle

```
Order.status:  escrow → released    (buyer confirms delivery, no open dispute)
                  ↳ refunded        (dispute resolved for buyer)
```

**Deposit** (`POST /market/orders`, `market.js:389`):

1. Validate listing is active, stock sufficient, buyer ≠ seller
2. Atomically reserve stock
3. Optional loyalty code claimed atomically (`unused → redeemed`)
4. `engine.transfer({ from: buyer, to: 'sw_system_escrow', amount: total, type: 'escrow:deposit' })`
5. Order created with `status: 'escrow'`
6. **Any failure triggers a full rollback**, including `escrow:rollback`

**Release** (`releaseEscrow`, `market.js:27-102`) — exactly-once via atomic claim:

```
Order.updateOne({ _id, status: 'escrow' }, { $set: { status: 'released', settledAt: Date.now() } });
if (modifiedCount !== 1) throw EngineError('ALREADY_SETTLED', ...);
```

**Real fee arithmetic** (`rules.js:129-134`), from our run:

|       |                 |    |     |     |   |          |       |             |
|-------|-----------------|----|-----|-----|---|----------|-------|-------------|
| total | 100 WELL @ 3%   | -> | fee | 3   | + | proceeds | 97    | (sum 100)   |
| total | 49.99 WELL @ 3% | -> | fee | 1.5 | + | proceeds | 48.49 | (sum 49.99) |
| total | 1000 WELL @ 3%  | -> | fee | 30  | + | proceeds | 970   | (sum 1000)  |

Then two transfers: `escrow:release` (escrow → seller, net) and `escrow:fee` (escrow → treasury). If the seller leg succeeds and the fee leg fails, a `escrow:release-reversal` compensates; if that *also* fails the order stays `released` and raises `SETTLE_STUCK` for manual reconciliation. That is a deliberate, documented choice — better a flagged stuck order than a silent double-spend.

**What to say:** "Escrow is genuinely correct. Settlement is exactly-once via a conditional database claim, not a read-then-write, so concurrent confirmation attempts can't pay twice. We found and fixed that bug ourselves — it's section 9 of our `glitch-hunt-web3.js` adversarial suite."

**Known limitations (Appendix A):** `cancelled` is declared in the enum but no route writes it; all buyers' funds sit in one fungible system wallet, so a pool shortfall would cross-subsidise another order.

## Feature 5 — Decentralized identity

Covered in §2.4. Each user gets an ed25519 keypair on first `/wallet` access; the address is deterministically derived from the public key.

## Feature 6 — User-owned health ledger

`POST /health/ledger/anchor` creates a snapshot. **Only the digest goes on-chain** — the payload stays private:

```
engine.anchor('health:anchor', ..., { secret: snapshot.payload, public: { digest, assessmentCount, intakeCount } })
```

`buildHealthDigest` covers: assessment `_id`, `createdAt`, `updatedAt`, `priority`, `intakeCount`, `latestAssessmentAt`. Reward: 3 WELL per day (capped by the `(user, kind, day)` unique index).

**Honest framing:** "The digest proves a snapshot existed at a point in time. It is not a full history record — the fields it covers are a subset of the assessment. And because nothing retains the earlier digests, it proves existence, not continuity." (Appendix A, item 6.)

## Feature 7 — Data sovereignty rewards

Users grant a research data share and receive 25 WELL. Consent is **real and revocable** (`data.js:357-378`): `DELETE /data/shares/:id` flips status, anchors a `consent:revoke` transaction, **and destroys the stored ciphertext**. That's better than most capstone projects manage — say so.

The dataset is coarse-banded server-side (`ageBand`, `assessmentCountBand`, `adherenceBand`, `streakBand`, `memberSinceYear`) — no ids, no dates, no free text.

**Honest framing:** "We call it an anonymised dataset, and the coarse banding is genuinely re-identification-resistant. But it's one user's own summary, not a cohort — there's no aggregation and no k-anonymity. The reward is for consenting to a study that can't yet retrieve the data, which is a real gap." (Appendix A, item 8.)

## Feature 8 — Decentralized encrypted storage

`POST /storage/pin` → encrypt (AES-256-GCM) → content-address → store `{ cid, ciphertext, iv, tag }`. `GET /storage/:cid` → decrypt for the owner, returns an integrity flag.

Uniqueness is `{ cid, owner }` — **per owner**, not global.

**Remember from §2.5: the CID identifies a specific encrypted blob, not the plaintext.** Two pins of identical content produce different CIDs because each encryption uses a fresh IV.

## Feature 9 — WELL token rewards

Every reward, with real arithmetic

| kind             | Amount                 | Source                                                                          | Idempotency key ( refId ) |
|------------------|------------------------|---------------------------------------------------------------------------------|---------------------------|
| welcome_bonus    | 100                    | welcomeBonus                                                                    | —                         |
| checkin          | 6 → 15 (streak-scaled) | $\text{rewardCheckin} + \text{rewardStreakStep} \times \min(\text{streak}, 10)$ | today's date              |
| intake           | 5                      | rewardIntakeDay                                                                 | today's date              |
| assessment       | 15                     | rewardAssessment                                                                | assessment _id            |
| knowledge        | 10                     | knowledgeReward                                                                 | post _id                  |
| knowledge-upvote | 2 (cap 40)             | knowledgeUpvoteReward                                                           | <postId>:<n>              |
| curator          | 1                      | curatorReward                                                                   | post _id                  |
| data-share       | 25                     | dataShareReward                                                                 | share _id                 |
| health-anchor    | 3                      | healthAnchorReward                                                              | today's date              |
| trial            | 60 / 45 / 30           | per-trial document value                                                        | trial _id                 |
| juror            | 5                      | jurorReward                                                                     | dispute _id               |

### Check-in streak — real output

$\text{checkinReward} = \text{rewardCheckin} + \text{rewardStreakStep} \times \min(\text{streak}, 10)$ , defaults 5 and 1 → **cap 15**.

| day | streak | amount |            |
|-----|--------|--------|------------|
| 1   | 1      | 6      |            |
| 2   | 2      | 7      |            |
| 3   | 3      | 8      |            |
| 4   | 4      | 9      |            |
| 5   | 5      | 10     |            |
| 6   | 6      | 11     |            |
| 7   | 7      | 12     |            |
| 8   | 8      | 13     |            |
| 9   | 9      | 14     |            |
| 10  | 10     | 15     |            |
| 11  | 11     | 15     | <-- capped |
| 12  | 12     | 15     |            |
| 13  | 13     | 15     |            |

Note day 1 pays **6, not 5** — the formula applies the first streak step immediately (`Math.max(prospective, 1)`).

**The streak-anchor rule** — why a gap resets to zero:

```
last check-in 2026-10-05, today 2026-10-12 -> streak 0
last check-in 2026-10-11, today 2026-10-12 -> streak 1
```

The most recent check-in must be **today or yesterday**. Miss one full day and the streak is gone.

**What to say:** "Idempotency is a unique database index on (user, kind, refId), not an application-level check. A retried request cannot double-mint — the insert fails and we return `alreadyClaimed`."

**What NOT to say:** don't claim the faucet is bounded. `refId` is a fresh id per post, per share, per assessment — so the anti-replay index prevents *replays* but not *volume*. That's Appendix A, item 7, and it's the first thing I'd fix.

## Feature 10 — Achievement NFTs

Eleven achievements (`rules.js:107-119`), minted when a stats threshold is met:

| kind           | Name                  | Metric          | Min |
|----------------|-----------------------|-----------------|-----|
| first-steps    | First Steps           | assessments     | 1   |
| week-warrior   | 7-Day Streak          | checkinStreak   | 7   |
| monthly-master | 30-Day Streak         | checkinStreak   | 30  |
| year-hero      | 365-Day Streak        | checkinStreak   | 365 |
| scholar        | Wellness Scholar      | assessments     | 10  |
| data-pioneer   | Data Pioneer          | dataShares      | 1   |
| governor       | Governor              | daoVotes        | 1   |
| merchant       | Market Participant    | ordersCompleted | 1   |
| diamond-hands  | Diamond Hands         | staked          | 500 |
| trailblazer    | Trailblazer           | trials          | 1   |
| verified-pro   | Verified Professional | bookings        | 1   |

Real run with `{assessments: 10, checkinStreak: 7, staked: 500}`:

```
week-warrior    7-Day Streak    metric=checkinStreak min=7 (have 7)
scholar         Wellness Scholar metric=assessments   min=10 (have 10)
diamond-hands   Diamond Hands    metric=staked        min=500 (have 500)
```

`tokenId = 'sw-' + sha256(kind:userId:serial)`; the `{ owner, kind }` unique index makes each achievement **soulbound** — one per user, forever, no transfer.

## Feature 11 — Staking for premium access

**Formula:** `round(staked × (apyPct/100) × (days/365))` — real output at 500 WELL, 12% APY:

```
1 day  -> 0.16 WELL
7 days -> 1.15 WELL
30 days -> 4.93 WELL
365 days -> 60.00 WELL
0% APY -> 0.00 WELL
```

**Accrual is lazy** — it happens on your next interaction ( `walletView`, `stake`, `/rewards/status` ), not on a cron. `accrueStake` requires  $\geq 1$  day elapsed and guards on the exact prior timestamp.

**Premium unlock** at `stakePremiumThreshold = 500`:

```
staked  0 -> unlocked=false progress=0
staked 250 -> unlocked=false progress=0.5
staked 500 -> unlocked=true  progress=1
staked 900 -> unlocked=true  progress=1
```

⚠️ **Critical:** `stakeApyPct` is DAO-governable, and `assertParamValue` bounds it to `[0, 1e9]` — not to a sensible economic range. A proposal setting it to  $10^9$  would pay  $\sim 10^7 \times$  per day. Combined with the quorum issue (Feature 13), one new account can do it. Say *"the bounds are numerical, not economic — that's item 2 in Appendix A"* rather than defending the cap.

## Feature 12 — Blockchain loyalty program

`POST /loyalty/redeem` burns WELL to `sw_system_treasury` and issues a one-time code `LOY-` + 10 hex chars. Minimum: `loyaltyRedeemRate` (10 WELL).

**Be precise:** "burn" here means transfer to the protocol treasury wallet — the tokens are not destroyed and supply is unchanged. It's a commitment mechanism, not deflationary." (Appendix A, item 9.)

## Feature 13 — DAO governance

**Proposal lifecycle:** `active` → `passed` | `rejected`. Three states total.

**Creation** requires `balance + staked > 0`, a valid `param` (own-property of `DEFAULT_PARAMS`), and passes `assertParamValue`. The proposer's own `for` vote is **auto-pushed** at creation (`govern.js:188`) — real, and worth disclosing.

**Voting** is guarded against TOCTOU races by re-asserting the deadline inside the atomic filter:

```
Proposal.updateOne(
  { _id, status: 'active', endsAt: { $gt: Date.now() }, 'votes.user': { $ne: req.user._id } },
  { $push: { votes: { user, address, choice, weight, at } } }
);
```

If `modifiedCount !== 1`, it re-reads to distinguish "voting has ended" from "you already voted." **That is careful work — show it.**

**Execution** happens in `finalizeExpired()`, latched to one sweep per 15s and capped at 25 proposals, because each passing proposal costs a mined block. It **claims first** (`{ _id, status: 'active' }`), then `setParam`, then appends `dao:execute`. One bad proposal is logged and skipped — it never aborts the sweep.

## The tally — real output

```
A: brand-new account self-votes (welcomeBonus=100)
  for=100 against=0 total=100 quorum=50 quorumReached=true PASSED=true voters=1

B: one 100 vs one 60 against
  for=100 against=60 total=160 quorum=50 quorumReached=true PASSED=true voters=2

C: 20 vs 20 (tie)
  for=20 against=20 total=40 quorum=50 quorumReached=false PASSED=false voters=2

D: 40 for only (below quorum 50)
  for=40 against=0 total=40 quorum=50 quorumReached=false PASSED=false voters=1

E: same wallet votes twice (100 then 100)
  for=100 against=0 total=100 quorum=50 quorumReached=true PASSED=true voters=1
```

Read A carefully: `welcomeBonus` (100) exceeds `daoQuorumWeight` (50). A brand-new account receives 100 WELL, opens a proposal, self-votes, and it passes.

**If a panel member spots this — and they will — the answer is:** "Yes, and it's the first thing I'd fix. The quorum is an absolute weight threshold set below the welcome bonus, so sybil resistance is effectively zero. A real DAO expresses quorum as a share of total supply, requires a proposal deposit, and has a timelock. I have this documented as item 1 in Appendix A with a proposed remedy: express quorum as a percentage of circulating supply, add a proposal deposit, and add a timelock before execution."

**Never** claim the DAO is secure against capture. E shows one-wallet-one-vote is enforced — say that positively, then pivot to the weight problem.

## Feature 14 — Community knowledge base

`POST /knowledge` publishes a post and pays 10 WELL. `POST /knowledge/:id/upvote` atomically pushes to `upvoters` and increments `upvotes`, paying 2 WELL to the author (capped at 40) and 1 WELL to the curator.

**Honest framing:** the upvote cap bounds *rewards*, not the `upvoters` array, which grows without limit. And two accounts can loop post → upvote indefinitely, each generating fresh payouts. The `refId` for upvotes includes the new upvote count, so each is a *new* key — anti-replay doesn't stop volume. Appendix A, item 7.

## Feature 15 — Decentralised dispute resolution

**Lifecycle:** `open → resolved`, with `outcome ∈ { buyer, seller, '' }`.

**Juror selection** (`market.js:148-158`) — verbatim:

```
Wallet.find({ isSystem: { $ne: true }, address: { $nin: excludeAddresses }, staked: { $gt: 0 } })
  .select('address').limit(5).lean();
```

Up to 5 staked, non-party wallets. No `.sort()` — so it's natural MongoDB order, i.e. insertion order. Front-runnable. The code comment says "Randomly selected jurors" (`market.js:143`) and the admin monitor reports "jurors are randomly drawn from staked holders" — **both are inaccurate**. Correct this yourself before anyone else does.

**If no staked wallet exists** → **open jury**: any non-party with `staked > 0` may vote, and `MIN_OPEN_JURY = 3` votes are required.

## Real verdicts

```
open jury, 2 buyer + 1 seller
  buyer=2 seller=1 cast=3 needed=3  complete=true  outcome="buyer"
open jury, only 2 votes cast
  buyer=2 seller=0 cast=2 needed=3  complete=false outcome="buyer"    <- incomplete, won't settle
CLOSED panel, 1 juror, 1 vote
  buyer=0 seller=1 cast=1 needed=1  complete=true  outcome="seller"    <- ONE person decides
closed panel, 5 jurors, 3 buyer (needed=3)
  buyer=3 seller=0 cast=3 needed=3  complete=true  outcome="buyer"
closed panel, 3 jurors, 1-1 tie
  buyer=1 seller=1 cast=2 needed=2  complete=true  outcome=""          <- tie, stays open
non-juror tries to vote
  buyer=1 seller=0 cast=1 needed=1  complete=true  outcome="buyer"    <- outsider IGNORED (correct)
```

Three things to demonstrate:

1. **The non-juror vote is correctly ignored** (last row) — juror eligibility is enforced.
2. **A tie produces** `complete: true` **with** `outcome: ''`, and `tryResolve` refuses to settle
3. **Row 3 is the vulnerability**: a closed panel of one juror needs one vote. The project's own

**And weight is never read.** `disputeOutcome` counts votes; it never touches `v.weight` — yet `market.js:632` computes it, the API returns it, and the UI renders `w {weight}` badges. The header comment calls it a "stake-weighted juror vote."

**Combined answer if challenged:** "Three honest limitations. First, juror selection has no `.sort()`, so it's insertion order, not random — the comment claiming randomness is wrong and I'll fix it. Second, a one-person panel decides alone. Third, stake weight is recorded and displayed but not used in the tally rule — the UI overstates it. The parts that are correct: eligibility is enforced, quorum-not-unanimity is the right rule, and ties correctly leave funds in escrow."

## Feature 16 — Interoperable health profile

Users issue a token (`randomBytes(16)` hex, TTL 1–720h). `GET /api/web3/share/:token` is public and unauthenticated and returns name, gender, age, health goals, symptoms, **medical conditions, current medications, and allergies** from the latest assessment.

⚠ **Handle this carefully.** A public endpoint returning medication lists is a privacy exposure regardless of token entropy. If asked: *"the token is 128 bits and revocable, but I agree an unauthenticated endpoint serving medication data is the wrong design — it should be scoped, authenticated, and auditable per view. That's Appendix A item 1."* Don't defend it.

`GET /health/export` issues the ed25519-signed credential from \$2.4.

## Feature 17 — Verifiable AI recommendations

Anchors `inputHash`, `outputHash`, `combinedHash` with `RECOMMENDER_LOGIC_VERSION = 'suppliwise-rec-engine@1'`.

Verification (`GET /recommendations/anchor/:assessmentId`) recomputes both digests and reports `{ inputIntact, outputIntact, recomputed, valid }`.

### Two honest limitations, state both:

1. `logicVersion` is a constant string. The actual recommender is an LLM whose provider and
2. The anchor is rewritable. The upsert has no "already anchored" guard, so editing an

*"The digest verification is real and reproducible. The version binding and re-anchoring guard are gaps I've documented."*

## Features 18, 19, 20 — Ecosystem

| Feature                   | Routes                                                                                     | Chain behaviour                                                                                                   |
|---------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 18 Clinical trial consent | <code>/trials</code> , <code>/trials/:id/optin</code> , <code>/withdraw</code>             | <code>trial:consent</code> / <code>trial:withdraw</code> anchors; 60/45/30 WELL reward. Withdrawal is idempotent. |
| 19 Oracle feeds           | <code>/oracle/feeds</code> , <code>/oracle/refresh</code>                                  | <code>oracle:update</code> anchor per changed feed; 60s latch                                                     |
| 20 Expert consultations   | <code>/experts</code> , <code>/experts/:id/book</code> , <code>/bookings/:id/cancel</code> | <code>consult:book</code> / <code>consult:refund</code> transfers                                                 |

**Booking flow** is careful: create the booking as `pending` first, then take payment; delete the unpaid booking if payment fails. Cancellation uses an atomic `confirmed → cancelled` claim, and reverts the claim if the refund fails.

### The oracle feeds — be blunt about these

Six feeds with hardcoded base values (`seed.js:85-92`): Vitamin D3 \$8.40, Magnesium \$12.90, Omega-3 \$15.60, WELL index \$0.12, 42 studies/7d, 99.2% verification rate.

Daily value derivation (`seed.js:108-114`):

```
const drift = (parseInt(sha256Hex(`${day}:${feed.key}`).slice(0, 8), 16) % 2000) / 1000;
```

A SHA-256 hash of the date and feed key, used as a deterministic pseudo-random walk. **This is auditable — it never changes retroactively — but it is not an oracle.** There is no Chainlink, no external feed, no attestation, no signature.

"These are deterministic simulated feeds. The property worth having is that a given day always produces the same value, so a historical anchor can be re-derived and checked. What's missing is any external data source and any signature. The UI calling them 'trusted real-world data signed daily' overstates it — I'll relabel that." The \$0.12 WELL index is read by exactly one consumer (a decorative marketplace stat tile) and **prices nothing**.

## Part 6 — Worked end-to-end scenarios

Trace these live during a demo. All numbers are real.

### Scenario A — A user's first 15 minutes

```
1. POST /api/auth/register          → user created
2. GET  /api/web3/wallet             → wallet auto-created
   ed25519 keypair generated, address = 0x<40 hex>
   privateKeyEnc = AES-256-GCM(privateKey, JWT_SECRET)
   grantReward('welcome_bonus', 100) → balance 100 [block 1]
   chain tx type: reward:welcome_bonus

3. POST /api/auth/login → token
4. Complete assessment          → assessment + aiResults
5. POST /api/web3/rewards/assessment → +15 WELL [block 2]
   grantReward('assessment', 15, refId=assessmentId)
   idempotent – replay returns alreadyClaimed, amount 0

6. POST /api/web3/rewards/checkin   → +6 WELL [block 3]
   streak 1 → 5 + 1×1 = 6

7. GET  /api/web3/chain             → see blocks 1,2,3
8. POST /api/web3/rewards/achievements/check
   → mints 'first-steps' NFT [block 4]
```

Final state: **balance 121 WELL**, 1 NFT, 4 blocks.

**Demo line:** "Four user actions produced four independent mined blocks, each hash-linked to the last. Let me show you the chain."

## Scenario B — Two users, one marketplace order

Alice (0xAAA) buys 2 × "Magnesium Glycinate" @ 60 WELL from the brand wallet  
total = 120 WELL

POST /market/orders

stock 35 → 33 (atomic reservation)  
Alice: 121 → 1 (transfer, escrow:deposit) [block 1]  
escrow: 0 → 120  
Order: status 'escrow' [block 2: escrow:deposit]  
↓ brand ships, Alice confirms

POST /market/orders/:id/confirm

escrowSplit(120, 3%) → fee 3.60, proceeds 116.40  
escrow → seller: 116.40 (escrow:release) [block 3]  
escrow → treasury: 3.60 (escrow:fee) [block 4]  
escrow balance: 0  
Order: escrow → released [atomic claim, exactly-once]

**Try to double-confirm:** the second call's `findOneAndUpdate({status: 'escrow'})` matches nothing → `ALREADY_SETTLED`. This is your best "we got this right" demonstration.

## Scenario C — A dispute

Order in escrow. Buyer opens a dispute (reason ≥ 10 chars).

Dispute created, status 'open' [block: dispute:open]  
selectJurors → up to 5 staked non-party wallets

If NO staked wallets exist → open jury, MIN\_OPEN\_JURY = 3 required

juror1 → buyer juror2 → buyer juror3 → seller  
buyer=2 seller=1 cast=3 needed=3 complete=true outcome="buyer"  
→ refundEscrow: escrow → buyer (full total) [block: escrow:refund]  
→ listing stock restored (33 → 35)  
→ each juror +5 WELL (jurorReward) [block: reward:juror]  
→ Dispute: open → resolved, outcome 'buyer' [atomic claim]

A tie (1-1-1) → outcome "" → tryResolve returns null → funds STAY in escrow

## Scenario D — Governance, including the failure mode

---

```
Alice (121 WELL) proposes: stakeApyPct = 15
  auto-vote: for, weight 121
Bob (60 WELL) votes against
  for=121 against=60 total=181 quorum=50 → PASSED
  endsAt = now + 3 days
  (finalizeExpired runs on GET /dao/proposals after endsAt)
  setParam('stakeApyPct', 15, 'dao:<id>')
  append dao:execute [block]

--- now do the same as a BRAND-NEW account ---
Fresh account: 100 WELL (welcome bonus)
  proposes: daoQuorumWeight = 0
  auto-vote: for, weight 100
  for=100 against=0 total=100 quorum=50 → PASSED ← ONE ACCOUNT
  Then every future proposal passes on a single self-vote.
```

**Demo this yourself, on purpose.** Volunteering the weakness is the strongest possible move — it proves you understand the system rather than just demonstrating it.

---

## Part 7 — Reference tables

### 7.1 DAO parameters ( `models/Web3.js:109-128` )

| Parameter                          | Default | Governable        |
|------------------------------------|---------|-------------------|
| <code>welcomeBonus</code>          | 100     | ✓                 |
| <code>rewardCheckin</code>         | 5       | ✓                 |
| <code>rewardStreakStep</code>      | 1       | ✓                 |
| <code>rewardAssessment</code>      | 15      | ✓                 |
| <code>rewardIntakeDay</code>       | 5       | ✓                 |
| <code>stakeApyPct</code>           | 12      | ✓                 |
| <code>stakePremiumThreshold</code> | 500     | ✓                 |
| <code>marketplaceFeePct</code>     | 3       | ✓                 |
| <code>loyaltyRedeemRate</code>     | 10      | ✓                 |
| <code>knowledgeReward</code>       | 10      | ✓                 |
| <code>knowledgeUpvoteReward</code> | 2       | ✓                 |
| <code>knowledgeUpvoteCap</code>    | 40      | ✓                 |
| <code>curatorReward</code>         | 1       | ✓                 |
| <code>dataShareReward</code>       | 25      | ✓                 |
| <code>jurorReward</code>           | 5       | ✓                 |
| <code>daoVotingDays</code>         | 3       | ✓ (bounded 1–365) |
| <code>daoQuorumWeight</code>       | 50      | ✓                 |
| <code>healthAnchorReward</code>    | 3       | ✓                 |

All 18 are DAO-governable, and all are bounded only by `[0, 1e9]` (except `daoVotingDays`). That is a numerical guard, not an economic one. Know it cold.

### 7.2 System wallets

| Address                            | Label                       |
|------------------------------------|-----------------------------|
| <code>sw_system_treasury</code>    | Protocol Treasury           |
| <code>sw_system_escrow</code>      | Smart-Contract Escrow       |
| <code>sw_system_brand</code>       | SuppliWise Verified Brands  |
| <code>sw_system_expert_pool</code> | Verified Professionals Pool |

All created with `balance: 0`. **All WELL that has ever existed came from the faucet** — there is no genesis allocation and no external liquidity.

## 7.3 Status enums

| Entity                                              | Values                                                                                                                                                                                      |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Order.status</code>                           | <code>escrow</code> , <code>released</code> , <code>refunded</code> , <code>cancelled</code> <i>(never written)</i>                                                                         |
| <code>Dispute.status</code> / <code>.outcome</code> | <code>open</code> , <code>resolved</code> / <code>buyer</code> , <code>seller</code> , <code>''</code>                                                                                      |
| <code>Proposal.status</code>                        | <code>active</code> , <code>passed</code> , <code>rejected</code>                                                                                                                           |
| <code>Listing.category</code>                       | <code>Vitamins</code> , <code>Minerals</code> , <code>Herbs</code> , <code>Protein</code> , <code>Probiotics</code> , <code>Other</code>                                                    |
| <code>KnowledgePost.type</code>                     | <code>review</code> , <code>research</code> , <code>story</code>                                                                                                                            |
| <code>DataShare.scope</code> / <code>.status</code> | <code>nutrition-outcomes</code> , <code>adherence-study</code> , <code>general-research</code> / <code>active</code> , <code>revoked</code>                                                 |
| <code>Booking.status</code>                         | <code>pending</code> , <code>confirmed</code> , <code>cancelled</code>                                                                                                                      |
| <code>supplyEvent.step</code>                       | <code>raw-sourcing</code> , <code>manufacturing</code> , <code>lab-testing</code> , <code>quality-release</code> , <code>distribution</code> , <code>retail</code> , <code>delivered</code> |
| <code>certification.type</code>                     | <code>lab-report</code> , <code>organic</code> , <code>non-gmo</code> , <code>third-party</code> , <code>gmp</code> , <code>other</code>                                                    |

## 7.4 Chain operation reference

| Chain action          | Emits                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>reward:*</code> | <code>reward:welcome_bonus</code> , <code>reward:checkin</code> , <code>reward:intake</code> , <code>reward:assessment</code> , <code>reward:knowledge</code> , <code>reward:knowledge-upvote</code> , <code>reward:curator</code> , <code>reward:data-share</code> , <code>reward:health-anchor</code> , <code>reward:trial</code> , <code>reward:juror</code> |
| Staking               | <code>stake</code> , <code>unstake</code> , <code>stake_reward</code>                                                                                                                                                                                                                                                                                           |
| Marketplace           | <code>market:list</code> , <code>escrow:deposit</code> , <code>escrow:rollback</code> , <code>escrow:release</code> , <code>escrow:fee</code> , <code>escrow:release-reversal</code> , <code>escrow:refund</code>                                                                                                                                               |
| Disputes              | <code>dispute:open</code>                                                                                                                                                                                                                                                                                                                                       |
| Governance            | <code>dao:propose</code> , <code>dao:vote</code> , <code>dao:execute</code>                                                                                                                                                                                                                                                                                     |
| Knowledge             | <code>knowledge:post</code>                                                                                                                                                                                                                                                                                                                                     |
| Supply                | <code>supply:create</code> , <code>supply:step</code> , <code>cert:anchor</code>                                                                                                                                                                                                                                                                                |
| Data                  | <code>health:anchor</code> , <code>storage:pin</code> , <code>consent:grant</code> , <code>consent:revoke</code> , <code>rec:anchor</code>                                                                                                                                                                                                                      |
| Ecosystem             | <code>trial:consent</code> , <code>trial:withdraw</code> , <code>oracle:update</code> , <code>consult:book</code> , <code>consult:refund</code>                                                                                                                                                                                                                 |
| Loyalty               | <code>loyalty:burn</code>                                                                                                                                                                                                                                                                                                                                       |

## Part 8 — The question bank

---

This is the part that earns marks. Practice saying these out loud.

### Tier 1 — Fundamentals

---

#### Q1. "Is this a real blockchain?"

No, and I want to be precise about what it is: a private, append-only, SHA-256 hash-chained audit ledger in our own MongoDB. Real cryptographic primitives, single writer, no consensus network. It's a tamper-evident audit log, not a distributed ledger. We chose that deliberately and documented the boundary.

#### Q2. "Then why call it blockchain?"

The properties we wanted are blockchain properties: append-only history, hash-linked blocks, cryptographic identity, signed credentials, verifiable audit trails. We implemented those properties in a form we could operate as a student project. The naming is aspirational; the cryptography is real.

#### Q3. "Show me consensus."

There isn't any — `ledger.js:199-221` is a promise queue that serialises appends in one process. Its job is to stop concurrent requests forking the chain, not to reach agreement between independent parties.

#### Q4. "Where are the nodes?"

One. One Node process, one MongoDB. If a panel member asks me to show two replicas diverging, I couldn't — and I'd list that as the first architectural limitation.

#### Q5. "What's the block time?"

There's no block interval. Blocks are produced on demand, one per state change, and each costs about 80ms of SHA-256 at difficulty 3. It's request-driven, not interval-driven.

#### Q6. "What's the transaction fee?"

There is no fee model. Anchoring is unmetered and free. I know that's a DoS surface — the mining cost is the only implicit throttle, and it's why `MAX_NONCE` is bounded.

### Tier 2 — Tamper evidence

---

#### Q7. "What stops someone editing history?"

Editing a `txHash` or `prevHash` breaks that block's recomputed hash and every block after it. Editing `txs[].data` does **not** break anything, because the header hash covers `txHash` strings rather than payload text. That's a real gap — the fix is to hash payloads into the header, which is a three-line change.

#### Q8. "Could you not just re-mine the whole chain?"

Yes. Whoever controls the database can rewrite everything, because the verification checkpoint is stored in that same database. That is the fundamental limitation of a single-custodian chain, and it's why I describe the system as tamper-evident, not tamper-proof.

#### Q9. "Why difficulty 3?"

1-in-4,096 odds, ~80ms. It's a bounded computational cost to make appending non-trivial without making the API unresponsive. It's not calibrated against network hashrate, because there is no network.

### Tier 3 — Tokens and governance

---

#### Q10. "Can someone capture the DAO?"

Yes, and I want to show you exactly how rather than have you find it. `welcomeBonus` is 100 and `daoQuorumWeight` is 50, so a brand-new account can self-vote a proposal through on its first day. Two fixes: express quorum as a share of total supply rather than an absolute number below the welcome bonus, and require a proposal deposit plus a timelock.

#### Q11. "Is the vote weight stake-weighted?"

No — and I should be clear that the UI overstates this. `disputeOutcome` and `tallyProposal` weight DAO votes by balance plus stake, but dispute resolution counts votes and ignores weight entirely, even though the weight is computed and displayed.

#### Q12. "Can someone farm tokens?"

Yes. `grantReward` is idempotent per `(user, kind, refId)`, which blocks *replays*, but each new post, share, or assessment is a new `refId`. The knowledge-base upvote path is the clearest loop: two accounts can alternate post → upvote indefinitely. A faucet cap per user per day is the fix.

#### Q13. "What backs the token?"

Nothing external. All four system wallets start at zero, there's no genesis allocation, and the only WELL in existence came from the faucet. There's no fiat leg and no withdrawal — the `$0.12` index feed is decorative and prices nothing.

#### Q14. "Who holds the private keys?"

The server does. Keys are generated server-side and stored AES-256-GCM-encrypted under `JWT_SECRET`, and the server signs with them. So signatures are real and verifiable, but this is not self-custody. Moving key generation to the client with WebCrypto is the real fix, and it would mean the server could no longer sign the health credential on the user's behalf.

## Tier 4 — Security and privacy

---

### Q15. "Is the health data encrypted?"

On-chain, yes — payloads are stored as digests, never in the clear, which is why the `dataHash` indirection exists. In our own database, no. `Assessment` stores medications and allergies as plaintext. We built a versioned AES-GCM envelope for TOTP secrets and didn't apply it to health data. That's a gap I want to close.

### Q16. "What's public without authentication?"

Three endpoints: batch verification, the QR generator, and shared health profiles. The third one returns medications and allergies behind a 128-bit token, and I now think it should be authenticated and per-view audited.

### Q17. "How do you prevent double-spending?"

Conditional atomic updates. `debit` is a single `findOneAndUpdate` filtered on `balance: { $gte: value }` — if funds are short, the filter doesn't match and nothing is written. There's no read-then-write window. Reward idempotency is a unique index. Settlement claims are conditional updates too. The one exception is `transfer()`, which is two writes with a

compensating rollback.

### Q18. "Is the paywall enforced server-side?"

Yes, at the router rather than the UI, via `requireFeature`. It's the strongest part of the project: plan-card feature bullets are *derived from the same gate definitions* the runtime enforces, so marketing copy can't drift from actual capability. The unknown-key path fails closed.

## Tier 5 — Engineering

---

### Q19. "How is the chain verified so cheaply?"

Two strategies. A deep `audit()` re-hashes every block and runs every five minutes. The default `verify()` does an incremental walk from a stored checkpoint, so it costs  $O(\text{new blocks})$ . The checkpoint tracks two separate things — how far integrity is proven, and when the last *deep* audit ran — because letting the incremental walk claim a full re-hash would be a lie.

### Q20. "What happens if verification times out?"

It returns `pending: true` with `valid: true`, because a check that hasn't come back yet is unknown, not broken. Reporting it as a failure caused a valid chain to display as "Some Warnings" after restarts.

### Q21. "How do you test this?"

Pure decision logic lives in `rules.js` with no I/O, so tallies, yields, streaks, and escrow splits are unit-tested directly. `glitch-hunt-web3.js` is a 21-section adversarial suite we wrote — it found the escrow double-confirm and stock

reservation races. `test-db-safety.test.js` adds a static scan that fails the build if any test calls `dropDatabase`, after a suite of ours deleted the live database twice.

## Q22. "What would you fix first?"

Four things, in order: express the DAO quorum as a share of supply; hash `txs[].data` into the block header; cap the faucet per user per day; and switch dispute juries to weighted, randomly ordered selection. All four are in Appendix A with specifics.

## Part 9 — Vocabulary cheat sheet

Say these correctly. Sloppy vocabulary makes correct work sound wrong.

| Term           | Say                                                    | Not                                 |
|----------------|--------------------------------------------------------|-------------------------------------|
| The chain      | "append-only hash-chained audit ledger"                | "blockchain", "Mainnet"             |
| Blocks         | "one block per state change, hash-linked"              | "mined by miners"                   |
| Difficulty     | "a bounded computational cost — 1 in 4,096"            | "network hashrate"                  |
| Tamper-evident | "edits break the hash chain"                           | "tamper-proof", "immutable"         |
| Identity       | "ed25519 keypair, address derived from the public key" | "self-custodied wallet"             |
| Keys           | "server-held, AES-256-GCM encrypted, exportable"       | "your keys, never touch our server" |
| Governance     | "token-weighted on-chain-style voting"                 | "decentralised DAO"                 |
| Market         | "P2P marketplace with escrow settlement"               | "verified marketplace"              |
| Oracles        | "deterministic simulated feeds"                        | "trusted real-world data, signed"   |
| Storage        | "encrypted, content-addressed blobs"                   | "decentralised IPFS storage"        |
| Consensus      | <b>do not use this word</b>                            | —                                   |

**One sentence that reframes everything if you need it:** *"We didn't build a blockchain. We built the part of one a supplement app actually needs — a verifiable cryptographic audit trail — and we can defend every line of it."*

## Part 10 — Live demo script

Nine minutes. Run it in this order so the strongest evidence lands first.

| # | Action                                                                                | What it proves                           |
|---|---------------------------------------------------------------------------------------|------------------------------------------|
| 1 | Open Block Explorer, click <b>Verify chain</b>                                        | Hash-linking works, live                 |
| 2 | Show a block, expand one tx, read its <code>txHash</code>                             | Real SHA-256, recomputable               |
| 3 | Read <code>dataHash</code> from two blocks and recompute one in a terminal            | "You can verify this yourself"           |
| 4 | Wallet panel → <b>Export key</b> → show the PEM                                       | Real ed25519; be honest it's server-held |
| 5 | Rewards → <b>check in</b> twice, show <code>alreadyClaimed</code>                     | Unique-index idempotency                 |
| 6 | Stake 500 WELL → show unlock progress at 100%                                         | Real threshold arithmetic                |
| 7 | Marketplace → buy → confirm → show <code>escrow → released</code> + two fee transfers | Exactly-once settlement                  |
| 8 | <b>Try confirming twice</b> → <code>ALREADY_SETTLED</code>                            | ★ Your strongest single demo             |
| 9 | Governance → create a proposal, vote from a second account                            | Real tally                               |

**Then, unprompted:** *"There's a sybil weakness in the DAO quorum I want to show you myself — let me create a fresh account and pass a proposal with it."* Volunteering this is worth more than any defence.

## Appendix A — Known limitations, stated plainly

**Bring this to the panel.** Naming your own weaknesses, with fixes, is the strongest move available in a capstone defence. Prioritised by how much a panel member can hurt you with it.

| #  | Limitation                                                                       | Impact                  | Fix                                                          |
|----|----------------------------------------------------------------------------------|-------------------------|--------------------------------------------------------------|
| 1  | Public <code>/share/:token</code> returns medications and allergies without auth | Privacy exposure        | Scope tokens, authenticate, per-view audit log               |
| 2  | <code>txs[].data</code> not in the header hash → payload edits undetected        | Weakens tamper-evidence | Hash payload into <code>headerHash</code> (3 lines)          |
| 3  | DAO quorum (50) < welcome bonus (100) → one account captures the DAO             | Governance integrity    | Quorum as % of supply + deposit + timelock                   |
| 4  | <code>stakeApyPct</code> bounded to 1e9 → unlimited APY                          | Tokenomics              | Per-parameter economic bounds                                |
| 5  | Server holds keys and signs on the user's behalf                                 | Not self-custody        | Client-side WebCrypto key generation                         |
| 6  | Faucet unbounded by volume (fresh <code>refId</code> per action)                 | Token inflation         | Per-user-per-day caps, slashing                              |
| 7  | Jurors selected without <code>.sort()</code> → insertion order                   | Dispute integrity       | Random sort / commit-reveal                                  |
| 8  | Dispute jury of 1 decides alone; weight ignored                                  | Dispute integrity       | <code>MIN_OPEN_JURY</code> for closed panels too; use weight |
| 9  | "Burn" is a transfer; supply unchanged                                           | Misleading              | Implement real burn or relabel                               |
| 10 | CID identifies a ciphertext, not plaintext                                       | Not content-addressed   | Hash plaintext, or fix the comment                           |
| 11 | <code>logicVersion</code> is a constant; anchors rewritable                      | Weak AI proof           | Record model+prompt hash; refuse re-anchor                   |
| 12 | Seeded demo data names real certification bodies                                 | Looks like fabrication  | Rename to <code>Demo Lab</code> , <code>SAMPLE</code>        |
| 13 | Health data plaintext in MongoDB                                                 | Data-at-rest            | Apply <code>secretBox</code> to health fields                |
| 14 | Single-process ledger — 2 instances can diverge                                  | No HA                   | Distributed lock or a real chain                             |
| 15 | <code>transfer()</code> two-write rollback can lose tokens on crash              | Fund loss               | MongoDB transaction or <code>client.close()</code>           |
| 16 | 5s verify budget returns <code>valid: true, pending: true</code>                 | Misleading status       | Report <code>pending</code> as unknown in the UI             |
| 17 | Chain grows forever; <code>audit()</code> is O(chain) every 5 min                | Performance cliff       | Prune or checkpoint permanently                              |
| 18 | <code>cancelled</code> order status unreachable                                  | Dead state              | Implement or remove from the enum                            |

Items 2, 3, 7, 8, and 12 are fixable in under an hour combined. Do them before the defence and this appendix becomes a roadmap instead of a confession.

## Appendix B — Verification script

Save as `verify-web3-demo.js` in `server/blockchain/` and run `node verify-web3-demo.js`. It requires only `crypto.js` and `rules.js` — no database, no network — and reproduces every figure in Parts 2, 3, 5, and 6.

```

// Reproduces every number and hash in the SuppliWise Web3 manual.
// Run: node verify-web3-demo.js (from server/blockchain/)
// Requires only the two pure modules. No DB, no network.

const crypto = require('crypto');
const C = require('./crypto');
const R = require('./rules');

// ledger.js:21-24 and :70-76 copied verbatim (avoids pulling in Mongoose models)
const DIFFICULTY = Math.max(1, Math.min(5, Number(process.env.CHAIN_DIFFICULTY) || 3));
const headerHash = ({ index, timestamp, prevHash, nonce, txs }) =>
  C.sha256Hex(C.stableStringify({
    index, timestamp, prevHash, nonce,
    txPart: (txs || []).map((t) => t.txHash).join('|'),
  }));

const normalizeTx = (tx) => {
  const timestamp = Number.isFinite(tx.timestamp) ? tx.timestamp : Date.now();
  const data = tx.data === undefined ? null : tx.data;
  const dataHash = tx.dataHash || C.hashPayload(data);
  return { txHash: C.sha256Hex(C.stableStringify({ type: tx.type, actor: tx.actor, dataHash, timestamp })),
    type: tx.type, actor: tx.actor, data, dataHash, timestamp };
};

// 1. Canonical JSON - key order must not change the bytes
const a = { amount: 25, kind: 'checkin', to: '0xabc' };
const b = { to: '0xabc', kind: 'checkin', amount: 25 };
console.log('stableStringify equal ?', C.stableStringify(a) === C.stableStringify(b));
console.log('naive JSON equal ? ', JSON.stringify(a) === JSON.stringify(b));

// 2. Payload hash - reproduce it by hand
const p = { amount: 100, reason: 'Wallet welcome bonus' };
console.log('dataHash ', C.hashPayload(p));
console.log('by hand ', crypto.createHash('sha256').update(C.stableStringify(p), 'utf8').digest('hex'));

// 3. Transaction hash
const tx = normalizeTx({ type: 'reward:welcome_bonus', actor: '0xDEMO', data: p });
console.log('txHash ', tx.txHash);

// 4. Mine a block, then tamper
const g = normalizeTx({ type: 'genesis', actor: 'sw:system',
  data: { network: 'suppliwise-mainnet', difficulty: DIFFICULTY, launchedAt: 0 } });
const header = { index: 0, timestamp: 1700000000000, prevHash: '0'.repeat(64), txs: [g] };
let found = null;
for (let nonce = 0; nonce <= 1 << 26; nonce += 1) {
  const h = headerHash({ ...header, nonce });
  if (h.startsWith('0'.repeat(DIFFICULTY))) { found = { nonce, hash: h }; break; }
}
console.log('nonce', found.nonce, 'hash', found.hash);
const tampered = headerHash({ ...header, timestamp: header.timestamp + 1, nonce: found.nonce });
console.log('tampered hash differs ?', tampered !== found.hash);

```

```

console.log('tampered still valid ? ', tampered.startsWith('0'.repeat(DIFFICULTY)));

// 5. Identity – sign and verify
const id = C.generateIdentity();
const msg = C.stableStringify({ assessment: 'A1', issuedAt: 0 });
const sig = C.sign(id.privateKey, msg);
console.log('address      ', id.address);
console.log('verify good   ', C.verify(id.publicKey, msg, sig));
console.log('verify tampered', C.verify(id.publicKey, msg + 'x', sig));

// 6. Encryption – and the CID claim, honestly
const enc = C.encrypt({'ageBand':"20-29"}, 'demo-secret');
console.log('decrypt ok    ', C.decrypt(enc, 'demo-secret'));
const cidA = C.contentId(enc.ct);
const cidB = C.contentId(C.encrypt({'ageBand':"20-29"}, 'demo-secret').ct);
console.log('same plaintext -> same CID ?', cidA === cidB, '(random IV => false)');

// 7. Escrow split
[100, 49.99, 1000].forEach((t) => {
  const r = R.escrowSplit(t, 3);
  console.log(`total ${t} -> fee ${r.fee} + proceeds ${r.proceeds}`);
});

// 8. Staking yield
[1, 7, 30, 365].forEach((d) =>
  console.log(`500 WELL @12% for ${d}d ->`, R.computeStakeReward(500, 12, d * 86400000)));

// 9. Check-in streak and cap
for (let d = 1; d <= 13; d += 1) {
  const k = `2026-10-${String(d).padStart(2, '0')}`;
  const s = R.checkinStreak([], k);
  console.log(`day ${d}: streak ${s}, amount ${R.checkinReward({ rewardCheckin: 5, rewardStreakStep: 1 }, Math.max(s, 1))}`);
}

// 10. Governance – including the capture case
console.log(R.tallyProposal([
  { address: '0xNEW', choice: 'for', weight: 100 }], { daoQuorumWeight: 50 }));

// 11. Disputes – including the one-juror case
console.log(R.disputeOutcome([
  { juror: '0xj1', choice: 'seller' }], ['0xj1']));

// 12. Staking unlocks and achievements
console.log(R.stakingUnlocks(250, { stakePremiumThreshold: 500 }));
console.log(R.eligibleAchievements({ assessments: 10, checkinStreak: 7, staked: 500 }, ['first-steps']));

```

**If a panel member wants proof, hand them this script.** Being able to reproduce your own claims in twenty seconds is the most persuasive thing in a defence.

---

Compiled from `server/blockchain/{crypto, ledger, engine, rules, seed}.js`, `server/models/Web3.js`, and `server/routes/web3/*`. All figures generated by executing the project's own modules on Node v24.19.0.